Съдържание

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за реализацията 1

1.1 1. Какво беше разработено . . . 1
1.2 2. Двигател на играта Покер Ледюк . . . 2
1.3 3. Реализации на алгоритмите . . . 2
1.4 4. Фаза на проучване (референция OpenSpiel) . . . 4
1.5 5. Еталонен тест — 180 s реално време . . . 5
1.6 6. Кръстосана проверка спрямо OpenSpiel . . . 6
1.7 7. Възпроизвеждане . . . 6

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за реализацията

Среда: Април 2026 **Игра:** Покер Ледюк (Leduc Poker, 6 карти, 2-ма играчи, 2 рунда на залагане) **Алгоритми:** Стандартен CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling **Цели:** CFR+ експлоатируемост < 0,001 в рамките на 180 s · кръстосана проверка спрямо OpenSpiel · log-log наклон ≈ −0,5 за MCCFR **Статус:** Всички цели постигнати

1.1 1. Какво беше разработено

Две линии на работа: **фаза на проучване**, използваща еталонните решители на OpenSpiel за изграждане на интуиция и установяване на истинни базови стойности, последвана от **реализация от нулата** на всички четири алгоритъма, написани на ръка на Python + само NumPy.

Структура на кода:

```
implementation/step03/
  cfr/
     leduc_poker.py             #                (6    , 2    ,       )
     info_set_node.py           #
     cfr_trainer.py             #       CFR (         ,            )
     cfrplus_trainer.py         # CFR+ (            +          +     )
     mccfr_external_trainer.py  # External Sampling MCCFR
     mccfr_outcome_trainer.py   # Outcome Sampling MCCFR ( -on-policy + IS)
     train.py                   #
     train_all_timed.py         #           180 s
  evaluate/
     best_response.py           #   -
     exploitability.py          # BR  + BR
```

```
    convergence.py            #
exploration/
    implDayOne1.py            #                    OpenSpiel
    leduc_comparison.py       #                     OpenSpiel
    leduc_race.py             # 5-
compare_openspiel.py           #                         OpenSpiel
utils/plotting.py              #
```

1.2 2. Двигател на играта Покер Ледюк

Пълна имплементация, съвпадаща със семантиката на OpenSpiel: 6 карти ({J, Q, K} × 2 цвята), два рунда на залагане с разкрита обща карта между тях, действия отказ/плащане/повишение, фиксирани размери на залозите (2 в рунд 1, 4 в рунд 2), максимум две повишения на рунд. Подреждане на ръцете на базата на двойка: скрита карта, съвпадаща с общата, бие всяка висока карта. Изброява всички 120 пермутации на раздаването за точно изчисление на очакваната стойност.

1.3 3. Реализации на алгоритмите

1.3.1 3.1 Стандартен CFR

Пълно обхождане на дървото с извадка от случайността по всичките 120 раздавания на итерация. Актуализациите на съжаленията се буферират по информационно множество и се прилагат атомарно в края на обхождането на всяко раздаване, за да се избегне дрейф на стратегията в средата на итерацията.

1.3.2 3.2 CFR+

Три модификации на стандартния CFR, всяка локализирана. Стъпката с ограничаването отдолу на съжаленията е ключовата промяна:

```
# cfrplus_trainer.py -                    (          ReLU)
for info_set, deltas in regret_buffer.items():
    node = node_map[info_set]
    for a in range(node.num_actions):
        node.regret_sum[a] = max(node.regret_sum[a] + deltas[a], 0.0)
```

Линейното осредняване на стратегиите претегля итерация `t` със самото `t` в текущата средна; редуващите се актуализации напредват само с един играч на преминаване (играч 0 при нечетни итерации, играч 1 при четни).

1.3.3 3.3 MCCFR External Sampling

Взема извадка от едно раздаване на итерация. Във възлите на обхождащия играч се изследват всички действия; във възлите на опонента се взема извадка от едно действие от текущата стратегия на опонента:

```
def external_cfr(state, update_player):
    if state.is_terminal():
        return state.get_utility(update_player)

    player = state.current_player()
    strategy = get_strategy(state.info_set(player))

    if player == update_player:
        #                 (              CFR)
        values = [external_cfr(state.apply(a), update_player)
                  for a in legal_actions]
        node_value = sum(strategy[a] * values[a] for a in legal_actions)
        for a in legal_actions:
            regret[info_set][a] += values[a] - node_value
        return node_value
    else:
        #
        sampled_action = sample(strategy)
        return external_cfr(state.apply(sampled_action), update_player)
```

Цена на итерация ≈ 42 възела (срещу 20 400 при пълно обхождане).

1.3.4 3.4 MCCFR Outcome Sampling

Взема извадка от една траектория от корен до терминал. Във възлите на обхождащия играч действията се теглят от смес ε-on-policy (ε = 0,6: равномерно с вероятност ε, иначе текущата стратегия). Актуализациите на съжалението се коригират чрез извадка по важност — съотношението между истинската вероятност за достигане и вероятността за извадка — което запазва свойството на неизместен оценител. Цена на итерация ≈ 5,5 възела.

1.3.5 3.5 Оценител на експлоатируемостта

Итеративен най-добър отговор с ограничение по информационно множество: за всеки играч изчислява оптималната контрастратегия при условие, че отговарящият трябва да играе еднакво действие по всички състояния в едно информационно множество. Връща `BR( ) + BR( )` като точна експлоати-

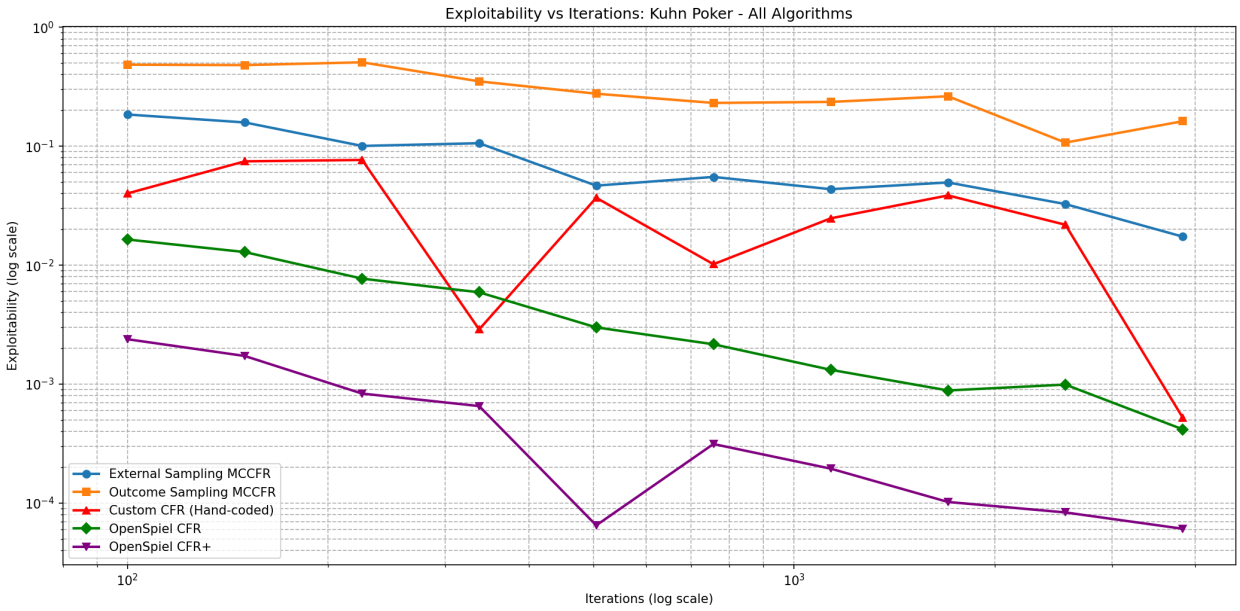
руемост.

1.4 4. Фаза на проучване (референция OpenSpiel)

1.4.1 4.1 Покер на Кун — 5000 итерации

Всички четири алгоритъма от OpenSpiel плюс персонализираният CFR от Стъпка 02, изпълнени за проверка на изправността при малък мащаб.

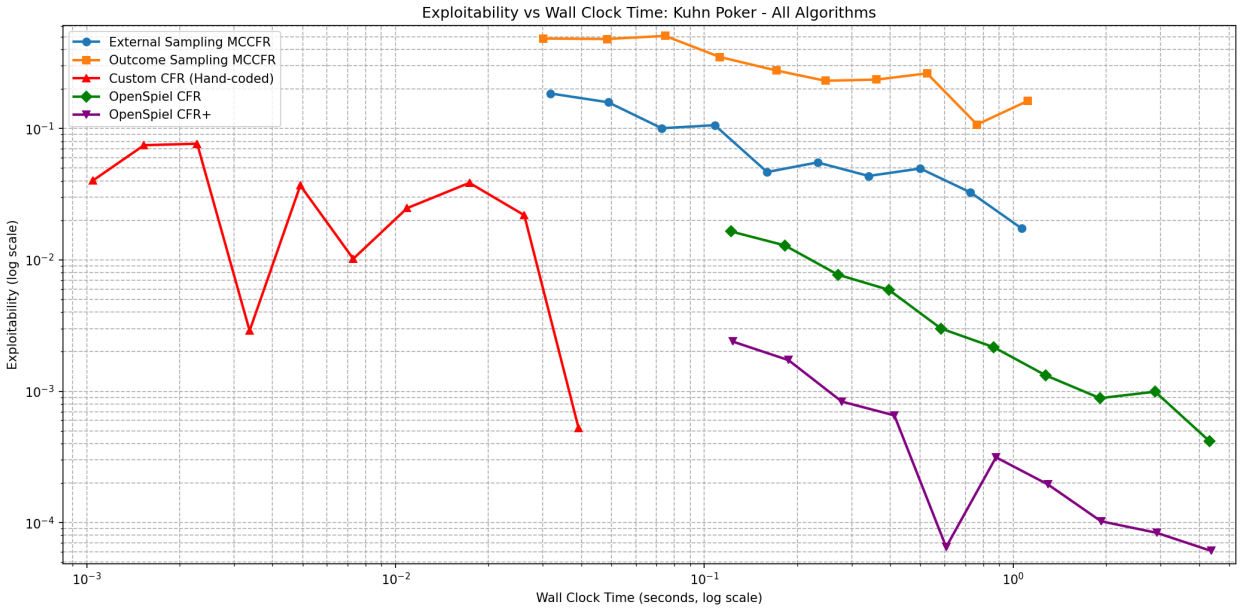
Алгоритъм	Експлоатируемост	Време
Персонализиран CFR (Стъпка 02)	~3,5×10 ^{4}	< 1 s
OpenSpiel CFR	~1,5×10 ^{3}	~2 s
CFR+	~3,0×10 ^{4}	~2 s
External Sampling MCCFR	~4×10 ^{3}	~1 s
Outcome Sampling MCCFR	~2,5×10 ^{2}	~1 s



Фигура 1: Кун — Експлоатируемост спрямо Итерации

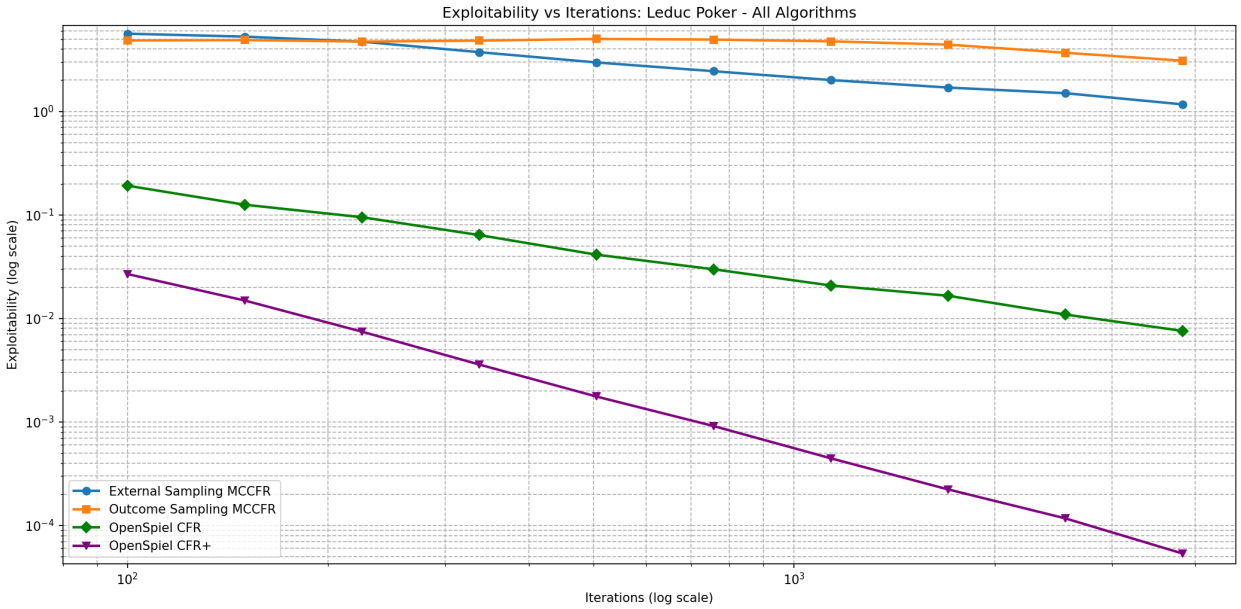
Всички алгоритми достигат почти Наш в рамките на секунди; разграничението е академично на този мащаб.

1.4.2 4.2 Покер Ледюк — 5000 итерации



Фигура 2: Кун — Експлоатируемост спрямо Реално време

Алгоритъм	Експлоатируемост @5k	Време
CFR+	~5,4×10 ^{5}	~859 s
Стандартен CFR	~7,6×10 ^{3}	~747 s
External Sampling MCCFR	~1,17	~7 s
Outcome Sampling MCCFR	~3,08	~5 s



Фигура 3: Ледюк — Експлоатируемост спрямо Итерации

На итерация методите с пълно обхождане доминират с 4–5 порядъка; в реално време разликата се свива, но не се затваря при дървета с размера на Ледюк.

Алгоритъм	Итерации	Крайна експлоатируемост	Инф. множества

CFR+ достига почти точно Наш ($2{,}6\times10^{-5}$) за 3 минути — над 150× по-добре от стандартния CFR, въпреки че изпълнява само пределно по-малко итерации. И двата варианта на MCCFR, въпреки милионите итерации, остават 3–4 порядъка по-зле при тази големина на играта, както предсказва анализът на компромиса дисперсия–скорост в резюмето.

1.6 6. Кръстосана проверка спрямо OpenSpiel

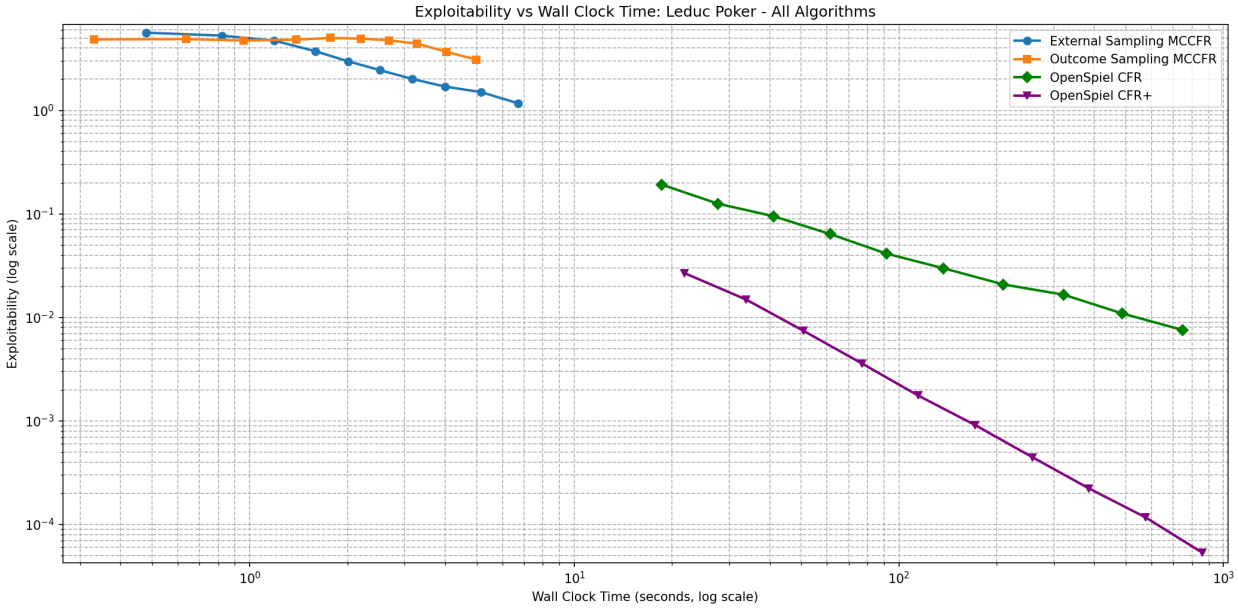
И двете имплементации с пълно обхождане бяха проверени спрямо еталонните решители на OpenSpiel при общ бюджет от 500 итерации.

Алгоритъм	Наш (500 итер.)	OpenSpiel (500 итер.)	Съвпадение
Стандартен CFR	0,020	0,022	
CFR+	$8{,}6\times10^{-4}$	$9{,}4\times10^{-4}$	

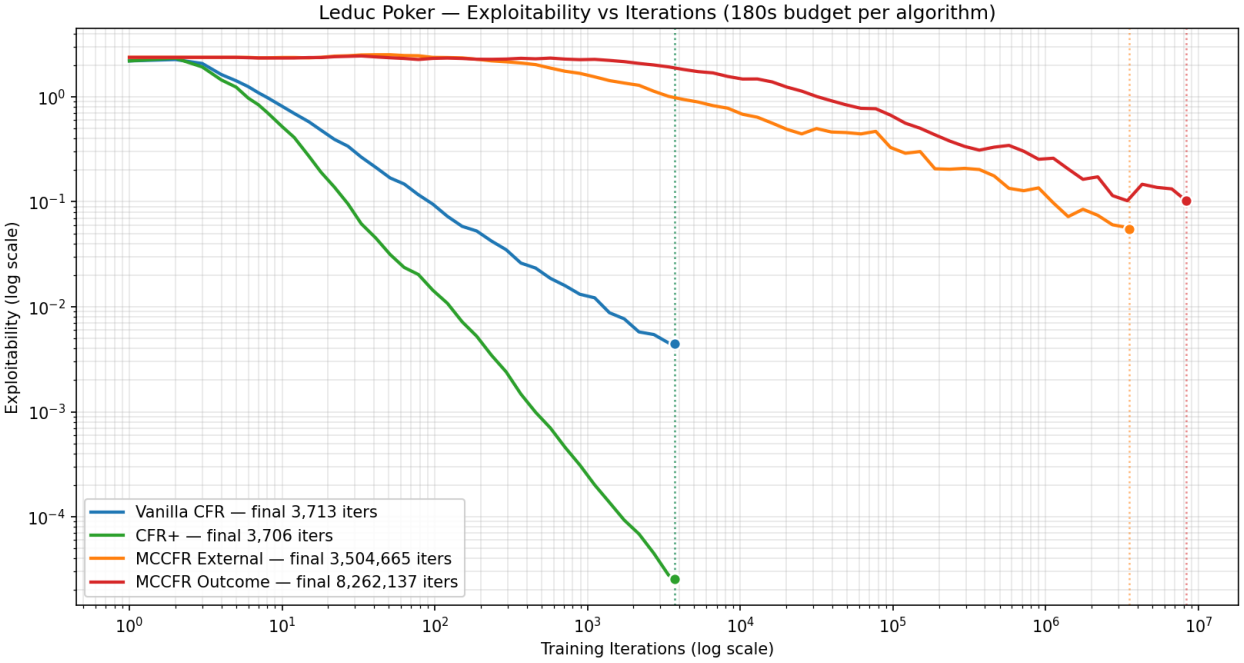
Малките разлики възникват от подредбата на раздаванията и случайните семена; и двата клонят към същото Равновесие на Наш в границите на шума.

1.7 7. Възпроизвеждане

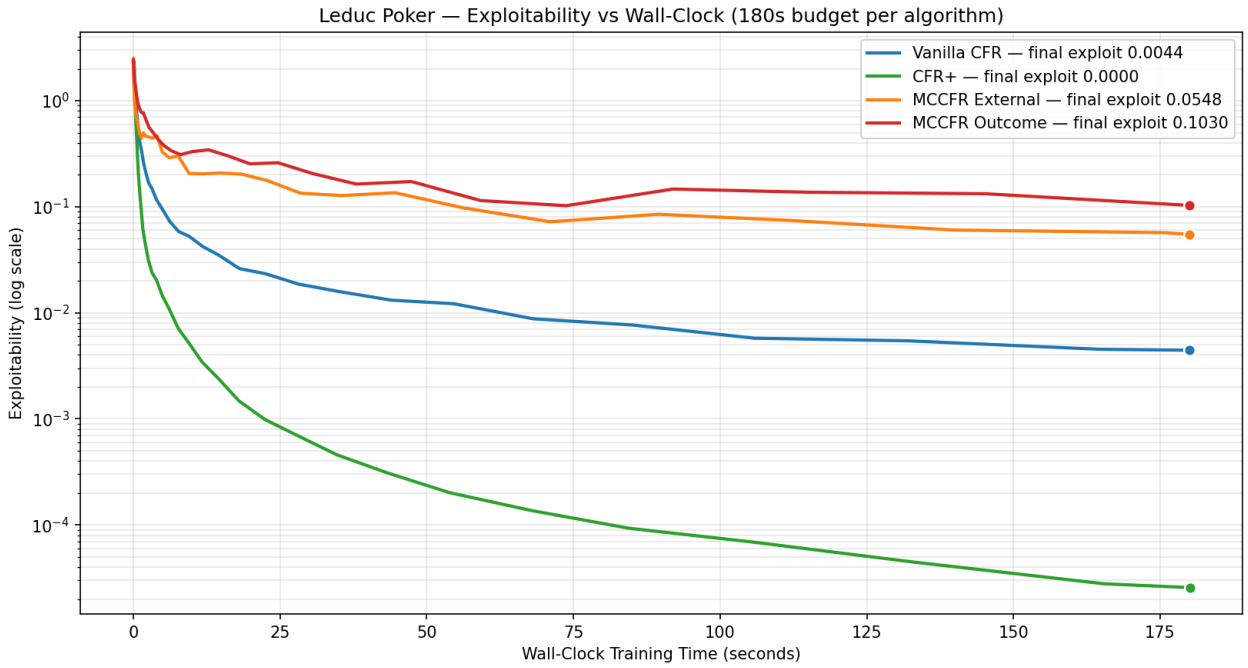
```
#                ,          .venv:

#                    (          train.py):
python implementation/step03/cfr/train.py

#          180 s                            :
python implementation/step03/cfr/train_all_timed.py

#       -                OpenSpiel:
python implementation/step03/exploration/implDayOne1.py      #
python implementation/step03/exploration/leduc_comparison.py #    ,
python implementation/step03/exploration/leduc_race.py       #    , 5-  .
```



Фигура 4: Ледюк — Експлоатируемост спрямо Реално време



Фигура 5: Експлоатируемост спрямо Итерации (log-log)



Фигура 6: Експлоатируемост спрямо Реално време (log-y)

```
# OpenSpiel:
python implementation/step03/compare_openspiel.py
```

Генерираните фигури са в `deliverables/reports/step03/figures/`.